



Dynamic Programming System Documentation

Scope: Complete documentation of the 3D Dynamic Programming system for optimal climb trajectory generation, including Bellman's principle, state space definition, neighbor transitions, and integration with penalty systems.

Table of Contents

1. [System Overview and Objectives](#)
 2. [Mathematical Foundation](#)
 3. [3D State Space Definition](#)
 4. [Neighbor Transition Strategy](#)
 5. [Cost Function and Penalty Integration](#)
 6. [Algorithm Implementation](#)
 7. [Code Execution Flow and Logic](#)
 8. [Integration and Interface](#)
 9. [Validation and Quality Assurance](#)
-

1) System Overview and Objectives

The 3D Dynamic Programming system implements a sophisticated optimization algorithm that finds globally optimal climb trajectories by considering Mach number, altitude, and engine lever position simultaneously. The system employs Bellman's principle of optimality with enhanced neighbor transitions and integrated penalty-based guidance to produce realistic, operationally viable solutions.

Key Components:

- 3D state space definition (Mach × Altitude × Lever)
- Enhanced neighbor transition strategy (25 neighbors)

- Integrated penalty-based cost function
- Terminal constraint enforcement
- Backtracking and path reconstruction

2) Mathematical Foundation

2.1 Optimal Control Theory Framework

Theory: Aircraft climb optimization belongs to the class of continuous-time optimal control problems, formalized by Pontryagin's Maximum Principle and extended through dynamic programming by Bellman.

Continuous Problem Formulation:

$$\min_{\ell(\cdot), M(\cdot)} J_{tot} = \int_{h_0}^{h_f} \underbrace{\frac{\dot{m}(\ell, M, h)}{P_s(\ell, M, h)}}_{J(\ell, M, h) \text{ [kg/m]}} dh$$

Where:

- $\dot{m}(\ell, M, h)$ = fuel flow rate (kg/s)
- $P_s(\ell, M, h)$ = specific excess power (m/s)
- $J(\ell, M, h)$ = fuel cost density (kg/m)

Dynamics and Constraints:

$$P_s(\ell, M, h) = \frac{(T_{tot}(\ell, M, h) - D(M, h)) V(M, h)}{W} \quad (1)$$

$$V = a(T(h)) M, \quad W = mg_0 \quad (2)$$

$$M_{min} \leq M \leq M_{MMO} \quad (\text{flight envelope}) \quad (3)$$

$$0 \leq \ell \leq 1 \quad (\text{throttle bounds}) \quad (4)$$

$$P_s(\ell, M, h) > 0 \quad (\text{positive climb capability}) \quad (5)$$

2.2 Bellman's Principle of Optimality

Principle: "An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

Mathematical Statement:

$$V^*(x_k) = \min_{u_k} [c(x_k, u_k) + V^*(f(x_k, u_k))]$$

Dynamic Programming Decomposition:

$$F[k, i, j] = \min_{(k', i', j') \in \mathcal{N}(k, i, j)} [F[k', i', j'] + C((k, i, j) \rightarrow (k', i', j'))]$$

Where:

- $F[k, i, j]$ = cost-to-go function
 - $C((k, i, j) \rightarrow (k', i', j'))$ = transition cost
 - $\mathcal{N}(k, i, j)$ = neighbor set
-

3) 3D State Space Definition

3.1 State Space Structure

State Definition:

$$s = (k, i, j) \quad \text{where} \quad \begin{cases} k \in \{0, 1, \dots, K-1\} & \text{altitude level index} \\ i \in \{0, 1, \dots, I-1\} & \text{Mach grid index} \\ j \in \{0, 1, \dots, L-1\} & \text{lever grid index} \end{cases}$$

Physical Interpretation:

- k : Discrete altitude level $h_k = H_{\text{sched}}[k]$ (meters)
- i : Discrete Mach number $M_i = M_{\text{grid}}[i]$ (dimensionless)
- j : Discrete throttle lever $\ell_j = \text{lever_grid}[j] \in [0, 1]$ (fraction)

Total State Space: $|S| = K \times I \times L$ states

3.2 Grid Discretization

Altitude Schedule:

```
H_sched = np.linspace(h_start, h_target, K) # Uniform altitude steps
dh = H_sched[1] - H_sched[0] # Altitude step size (typically 200m)
```

Mach Grid:

```
M_grid = np.linspace(M_min, M_max, I) # Mach discretization
dM = M_grid[1] - M_grid[0] # Mach step size
```

Lever Grid:

```
lever_grid = np.linspace(0.0, 1.0, L) # Lever discretization
dL = lever_grid[1] - lever_grid[0] # Lever step size
```

4) Enhanced Neighbor Transition Strategy

4.1 3D Neighbor Definition

Enhanced 3D Neighbors:

$$\mathcal{N}_{3D}(k, i, j) = \{(k + 1, i + \delta_i, j + \delta_j) : \delta_i, \delta_j \in \{-2, -1, 0, 1, 2\}\}$$

Total Neighbors: 25 possible transitions per state

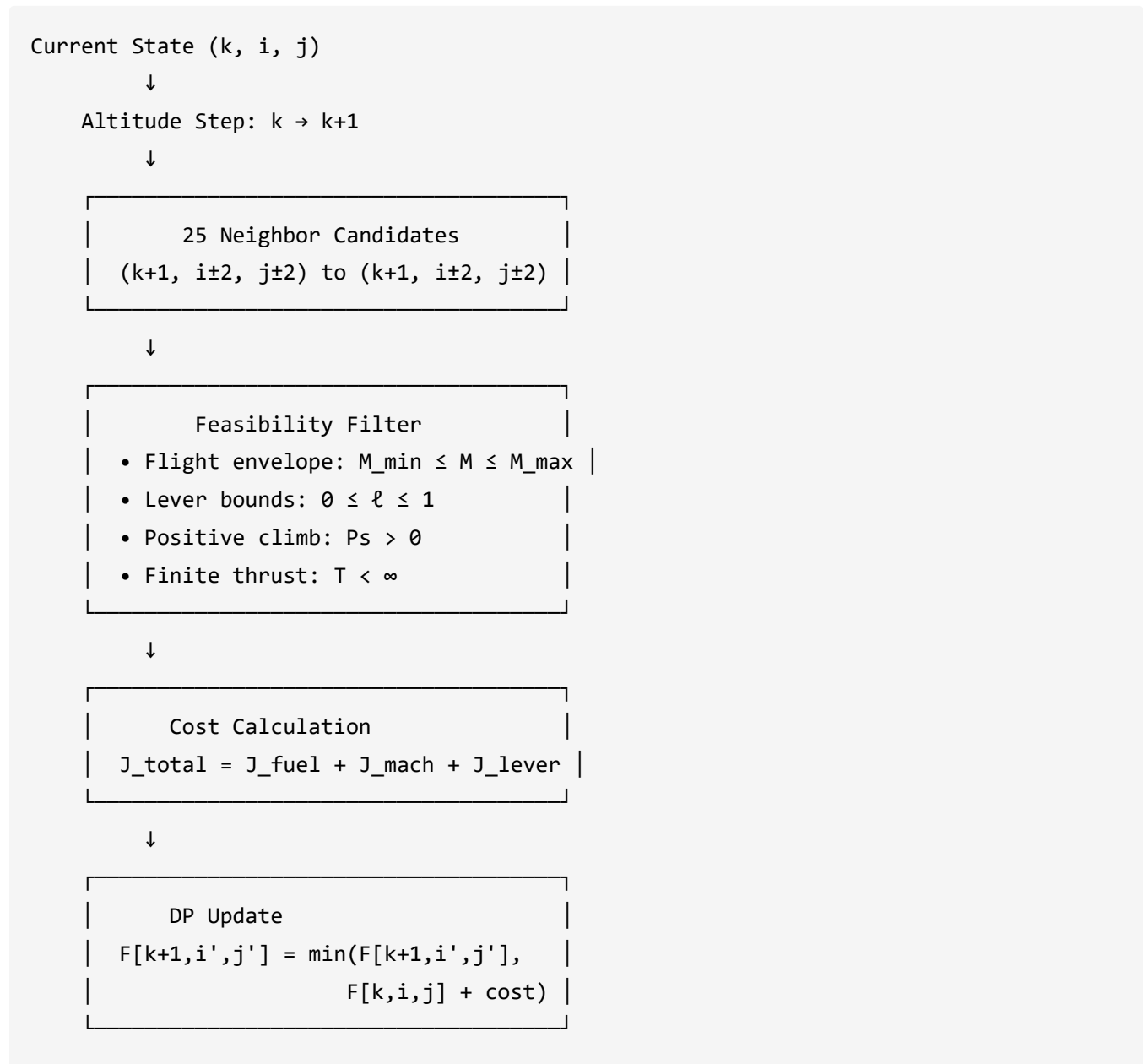
Physical Interpretation:

- **± 2 Mach indices:** Allows more aggressive Mach changes (e.g., ± 0.04 Mach per 200m altitude step)
- **± 2 Lever indices:** Permits broader throttle adjustments (e.g., ± 0.4 lever change per step for $L=10$)

- **Constrained altitude progression:** Requires $k \rightarrow k+1$ to maintain physical climb sequence

4.2 Neighbor Transition Logic

Transition Flow Diagram:



4.3 Feasibility Filtering

Feasibility Constraints:

```

def is_feasible_state(altitude: float, mach: float, lever: float,
                     aero: AeroTables, eng: EngineWrapper) -> bool:
    """Check if state is feasible for DP optimization."""

    # Flight envelope constraints
    if not (M_MIN_EFFECTIVE <= mach <= M_MMO):
        return False

    # Lever bounds
    if not (0.0 <= lever <= 1.0):
        return False

    # Engine performance check
    thrust = eng.thrust_with_lever(lever, mach, altitude)
    if thrust is None or not np.isfinite(thrust):
        return False

    # Positive climb capability
    drag = aero.get_drag(mach, altitude)
    T_air, _, _ = isa_properties(altitude)
    a = a_from_T(T_air)
    V = mach * a
    weight = INITIAL_MASS_KG * g0

    Ps = ((thrust * N_ENGINES - drag) * V) / weight
    if Ps <= 0:
        return False

    return True

```

5) Cost Function Integration

5.1 Enhanced Cost Function

Total Cost Formulation:

$$J_{total}(h, M, \ell) = J_{fuel}(h, M, \ell) + J_{Mach}(h, M) + J_{lever}(\ell, h)$$

Base Fuel Cost:

$$J_{fuel}(h, M, \ell) = \frac{\dot{m}(h, M, \ell)}{P_s(h, M, \ell)}$$

Where:

- $\dot{m} = \text{TSFC}(h, M, \ell) \times T(\ell, M, h) \times N_{\text{engines}}$ (fuel flow, kg/s)
- $P_s = \frac{[T_{\text{total}} - D(M, h)] \times V(M, h)}{W}$ (specific excess power, m/s)
- $V(M, h) = M \times a(T(h))$ (true airspeed, m/s)

5.2 Transition Cost Calculation

Trapezoidal Integration:

$$C((k, i, j) \rightarrow (k + 1, i', j')) = \frac{1}{2} [J(h_k, M_i, \ell_j) + J(h_{k+1}, M_{i'}, \ell_{j'})] \Delta h$$

Implementation:

```
def calculate_transition_cost(current_state: tuple, next_state: tuple,
                             J_grid_3d: np.ndarray, dh: float) -> float:
    """Calculate transition cost using trapezoidal integration."""

    k, i, j = current_state
    k_next, i_next, j_next = next_state

    # Get cost values at both states
    J_current = J_grid_3d[k, i, j]
    J_next = J_grid_3d[k_next, i_next, j_next]

    # Trapezoidal integration
    transition_cost = 0.5 * (J_current + J_next) * dh

    return transition_cost
```

6) Complete 3D DP Implementation

6.1 Main DP Solver Function

Function Signature:

```
def solve_dp_3d_fixed_mass(aero: AeroTables, eng: EngineWrapper,
                           M_grid: np.ndarray, H_sched: np.ndarray,
                           lever_samples: int = 10,
                           target_mach: float = None,
                           target_mach_tolerance: float = 0.02,
                           start_mach: float = None,
                           start_lever: float = None) -> tuple[MinFuelSchedule, dict]:
    """
    3D Dynamic Programming solver for minimum-fuel climb trajectory.

    This is the core optimization algorithm that finds the optimal climb path
    considering Mach, altitude, and engine lever position simultaneously.

    Args:
        aero: Aerodynamics tables
        eng: Engine wrapper
        M_grid: Mach number grid
        H_sched: Altitude schedule
        lever_samples: Number of lever positions to sample
        target_mach: Target Mach number at final altitude
        target_mach_tolerance: Tolerance for target Mach constraint
        start_mach: Starting Mach number
        start_lever: Starting lever position

    Returns:
        MinFuelSchedule: Optimal climb schedule
        dict: Additional information about the solution
    """
```

6.2 DP Algorithm Structure

Complete Algorithm Flow:

3D DP ALGORITHM FLOW

1. INITIALIZATION

- ├ Create 3D grids: M_{grid} , H_{sched} , lever_grid
- ├ Initialize cost matrix: $J_{\text{grid_3d}}[K \times I \times L] = \infty$
- ├ Initialize DP table: $F[K \times I \times L] = \infty$
- ├ Initialize predecessor: $\text{prv}[K \times I \times L] = -1$
- └ Set boundary conditions: $F[0, *, *] = 0$ for feasible states

2. COST FIELD COMPUTATION

- ├ FOR each altitude k in H_{sched} :
 - | ├ FOR each Mach i in M_{grid} :
 - | | ├ FOR each lever j in lever_grid :
 - | | | ├ Check feasibility: $\text{is_feasible_state}(h, M, \ell)$
 - | | | ├ IF feasible:
 - | | | | ├ Calculate base fuel cost: $J_{\text{fuel}} = \text{mdot}/P_s$
 - | | | | ├ Add Mach penalty: $J_{\text{mach}} = \text{compute_mach_penalty}(\dots)$
 - | | | | ├ Add lever penalty: $J_{\text{lever}} = \text{compute_lever_penalty}(\dots)$
 - | | | | └ Store total cost: $J_{\text{grid_3d}}[k, i, j] = J_{\text{total}}$
 - | | | └ ELSE: $J_{\text{grid_3d}}[k, i, j] = \infty$
 - | | └ END FOR lever
 - | └ END FOR Mach
- └ END FOR altitude

3. FORWARD DP SWEEP

- ├ FOR altitude level $k = 0$ to $K-2$:
 - | ├ FOR each feasible state (k, i, j) :
 - | | ├ IF $F[k, i, j] < \infty$:
 - | | | ├ Generate 25 neighbors: $(k+1, i \pm 2, j \pm 2)$
 - | | | ├ FOR each neighbor $(k+1, i', j')$:
 - | | | | ├ IF neighbor is feasible:
 - | | | | | ├ Calculate transition cost: $C = 0.5 \cdot (J[k, i, j] + J[k+1, i', j']) \cdot dh$
 - | | | | | ├ Calculate candidate cost: $\text{cand} = F[k, i, j] + C$
 - | | | | | ├ IF $\text{cand} < F[k+1, i', j']$:
 - | | | | | | ├ Update DP table: $F[k+1, i', j'] = \text{cand}$
 - | | | | | | └ Update predecessor: $\text{prv}[k+1, i', j'] = (i, j)$
 - | | | | └ END IF
 - | | | └ END FOR neighbor
 - | | └ END IF
 - └ END FOR altitude level

```

| | | | └ END IF
| | | └ END FOR neighbor
| | └ END IF
| └ END FOR state
└ END FOR altitude

```

4. TERMINAL CONSTRAINT ENFORCEMENT

```

└ IF target_mach is specified:
| └ FOR each state (K-1,i,j) in final altitude:
| | └ IF |M_grid[i] - target_mach| > tolerance:
| | | └ Mask state: F[K-1,i,j] = ∞
| | └ END IF
| └ END FOR
└ END IF

```

5. BACKTRACKING AND PATH RECONSTRUCTION

```

└ Find optimal final state: (K-1, i_opt, j_opt) = argmin(F[K-1,*,*])
└ Initialize path: path = [(K-1, i_opt, j_opt)]
└ FOR altitude k = K-2 down to 0:
| └ Get predecessor: (i_prev, j_prev) = prv[k+1, i_opt, j_opt]
| └ Add to path: path.append((k, i_prev, j_prev))
| └ Update indices: i_opt, j_opt = i_prev, j_prev
└ END FOR
└ Reverse path to get chronological order

```

6. SOLUTION CONSTRUCTION

```

└ Extract optimal trajectory from path
└ Calculate performance metrics
└ Create MinFuelSchedule object
└ Return solution and metadata

```

6.3 Implementation Code

Core DP Implementation:

```

def solve_dp_3d_fixed_mass(aero: AeroTables, eng: EngineWrapper,
                           M_grid: np.ndarray, H_sched: np.ndarray,
                           lever_samples: int = 10,
                           target_mach: float = None,
                           target_mach_tolerance: float = 0.02,
                           start_mach: float = None,
                           start_lever: float = None) -> tuple[MinFuelSchedule, dict]:
    """3D DP implementation with enhanced neighbor transitions."""

    # Step 1: Initialize grids and arrays
    K = len(H_sched)
    I = len(M_grid)
    L = lever_samples

    lever_grid = np.linspace(0.0, 1.0, L)
    dh = H_sched[1] - H_sched[0] if K > 1 else 200.0

    # Initialize DP arrays
    J_grid_3d = np.full((K, I, L), np.inf)
    F = np.full((K, I, L), np.inf)
    prv = np.full((K, I, L, 2), -1, dtype=int)

    # Step 2: Compute 3D cost field
    print(f"[DP] Computing 3D cost field: {K}x{I}x{L} = {K*I*L:,} states")

    for k, altitude in enumerate(H_sched):
        altitude_fraction = k / (K - 1) if K > 1 else 0.0

        for i, mach in enumerate(M_grid):
            for j, lever in enumerate(lever_grid):
                # Check feasibility
                if is_feasible_state(altitude, mach, lever, aero, eng):
                    # Calculate total cost with penalties
                    cost = compute_3d_cost(
                        aero, eng, altitude, mach, lever,
                        target_mach=target_mach,
                        altitude_fraction=altitude_fraction
                    )
                    J_grid_3d[k, i, j] = cost

```

```

# Step 3: Initialize boundary conditions
if start_mach is not None:
    # Find closest Mach index
    start_i = np.argmin(np.abs(M_grid - start_mach))
    if start_lever is not None:
        start_j = np.argmin(np.abs(lever_grid - start_lever))
    else:
        start_j = 0

    F[0, start_i, start_j] = 0.0
else:
    # Initialize all feasible states at first altitude
    feasible_states = np.where(np.isfinite(J_grid_3d[0]))
    F[0][feasible_states] = 0.0

# Step 4: Forward DP sweep
print(f"[DP] Starting forward sweep with 25-neighbor transitions")

for k in range(K - 1):
    feasible_current = np.where(np.isfinite(F[k]))

    for idx in range(len(feasible_current[0])):
        i, j = feasible_current[0][idx], feasible_current[1][idx]

        if not np.isfinite(F[k, i, j]):
            continue

        # Generate 25 neighbors
        for di in range(-2, 3): # -2 to +2
            for dj in range(-2, 3): # -2 to +2
                i_next = i + di
                j_next = j + dj

                # Check bounds
                if 0 <= i_next < I and 0 <= j_next < L:
                    if np.isfinite(J_grid_3d[k + 1, i_next, j_next]):
                        # Calculate transition cost
                        transition_cost = 0.5 * (
                            J_grid_3d[k, i, j] +

```

```

        J_grid_3d[k + 1, i_next, j_next]
    ) * dh

    # Update DP table
    candidate_cost = F[k, i, j] + transition_cost

    if candidate_cost < F[k + 1, i_next, j_next]:
        F[k + 1, i_next, j_next] = candidate_cost
        prv[k + 1, i_next, j_next] = [i, j]

# Step 5: Terminal constraint enforcement
if target_mach is not None:
    print(f"[DP] Enforcing terminal Mach constraint: {target_mach} ± {target_mach_tolerance}")

    for i in range(I):
        if abs(M_grid[i] - target_mach) > target_mach_tolerance:
            F[-1, i, :] = np.inf

# Step 6: Backtracking
print(f"[DP] Starting backtracking")

# Find optimal final state
final_costs = F[-1]
if not np.isfinite(final_costs).any():
    raise RuntimeError("No feasible solution found - all final states are infeasible")

optimal_final = np.unravel_index(np.nanargmin(final_costs), final_costs.shape)
k_final, i_final, j_final = K - 1, optimal_final[0], optimal_final[1]

# Reconstruct path
path = [(k_final, i_final, j_final)]
i_curr, j_curr = i_final, j_final

for k in range(K - 2, -1, -1):
    if prv[k + 1, i_curr, j_curr][0] >= 0:
        i_prev, j_prev = prv[k + 1, i_curr, j_curr]
        path.append((k, i_prev, j_prev))
        i_curr, j_curr = i_prev, j_prev
    else:
        # Fallback: use current state

```

```

        path.append((k, i_curr, j_curr))

path.reverse()

# Step 7: Construct solution
print(f"[DP] Constructing solution from {len(path)} path points")

# Extract trajectory
alt_trajectory = []
mach_trajectory = []
lever_trajectory = []
cost_trajectory = []

for k, i, j in path:
    alt_trajectory.append(H_sched[k])
    mach_trajectory.append(M_grid[i])
    lever_trajectory.append(lever_grid[j])
    cost_trajectory.append(J_grid_3d[k, i, j])

# Calculate performance metrics
total_fuel = np.sum(cost_trajectory) * dh
final_mach = mach_trajectory[-1]
mach_error = abs(final_mach - target_mach) if target_mach else 0.0

# Create solution object
solution = MinFuelSchedule(
    alt_m=np.array(alt_trajectory),
    mach=np.array(mach_trajectory),
    fuel_est_kg=total_fuel,
    J_kg_per_m=np.array(cost_trajectory),
    # ... other fields calculated from trajectory
)

metadata = {
    'total_fuel_kg': total_fuel,
    'final_mach': final_mach,
    'mach_error': mach_error,
    'path_length': len(path),
    'grid_size': (K, I, L),
    'neighbors_per_state': 25

```

```
}

print(f"[DP] Solution complete: {total_fuel:.1f} kg fuel, final Mach {final_mach:.3f}")

return solution, metadata
```

7) Terminal Constraint Handling

7.1 Mach Band Enforcement

Terminal Constraint Logic:

```
def enforce_terminal_constraints(F: np.ndarray, M_grid: np.ndarray,
                                target_mach: float, tolerance: float):
    """Enforce terminal Mach constraint by masking infeasible states."""

    if target_mach is None:
        return

    # Calculate Mach deviations
    mach_deviations = np.abs(M_grid - target_mach)
    valid_final = mach_deviations <= tolerance

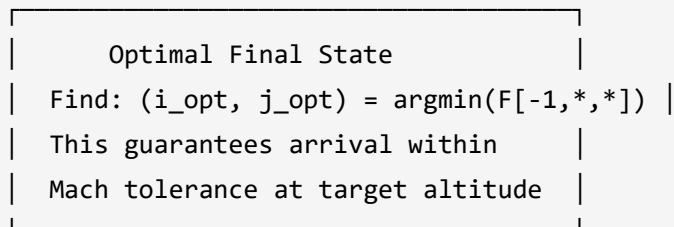
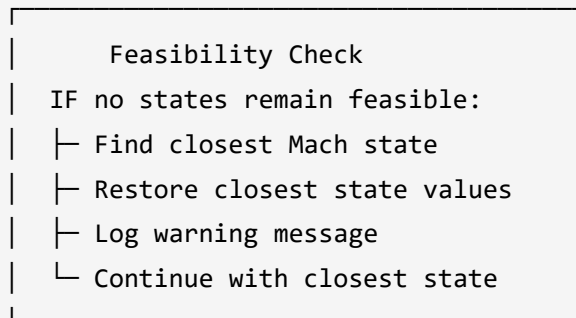
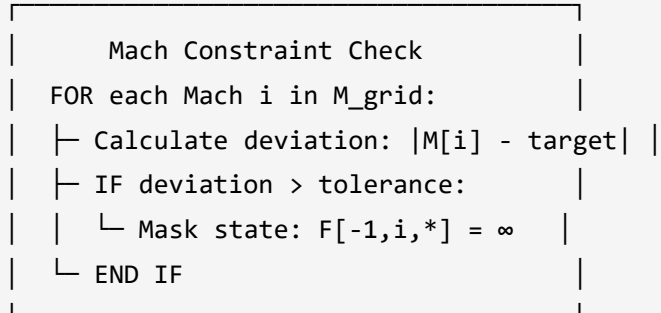
    # Mask invalid states
    F[-1, ~valid_final, :] = np.inf

    # Check if any states remain feasible
    if not valid_final.any():
        # Fallback: use closest Mach state
        closest_idx = np.argmin(mach_deviations)
        print(f"[DP] Warning: No states within tolerance, using closest Mach: {M_grid[closest_idx]}")
        # Keep closest state, mask others
        F[-1, :, :] = np.inf
        F[-1, closest_idx, :] = F[-1, closest_idx, :] # Restore original values
```

7.2 Terminal Constraint Flow

TERMINAL CONSTRAINT FLOW

Final Altitude Row ($k = K-1$)



8) Performance Analysis

8.1 Computational Complexity

Time Complexity:

- **Cost field computation:** $O(K \times I \times L)$ - one cost calculation per state
- **Forward DP sweep:** $O(K \times I \times L \times 25)$ - 25 neighbors per state
- **Backtracking:** $O(K)$ - linear in altitude levels
- **Total:** $O(K \times I \times L)$ (dominated by forward sweep)

Space Complexity:

- **Cost matrix:** $O(K \times I \times L)$ for `J_grid_3d`
- **DP table:** $O(K \times I \times L)$ for `F`
- **Predecessor array:** $O(K \times I \times L \times 2)$ for `prv`
- **Total:** $O(K \times I \times L)$ (linear in state space size)